# Introduction to Graphs

## Representations, Traversals, and Structural Patterns

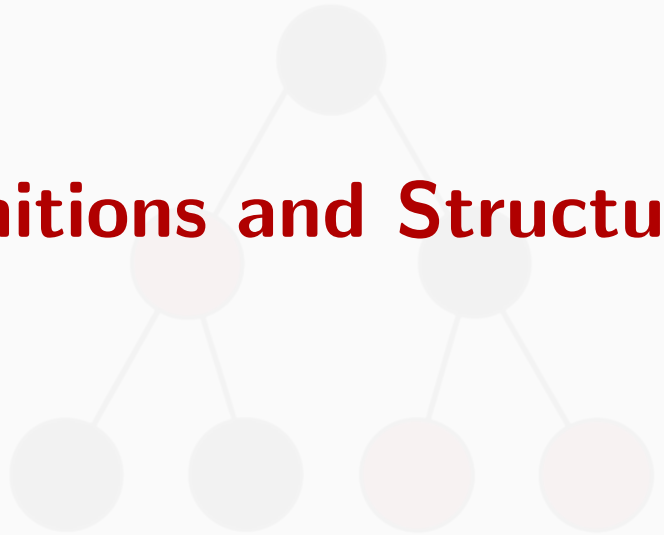
King's Competitive Programming Club

Workshop 4

# Workshop Overview


1. Graph Definitions and Structural Families
2. Storing Graphs: Representations and Trade-Offs
3. Breadth-First Search (BFS) and Unweighted Shortest Paths
4. Depth-First Search (DFS) and Edge Classification
5. Core Algorithmic Patterns for the Problem Sheet
6. Conclusion and Try-It-Yourself Solutions

# Graph Definitions and Structural Families



# What Is a Graph?

A **graph** $G = (V, E)$ consists of:

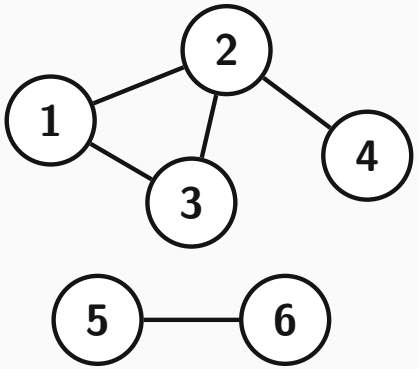
- → A set of **vertices** (nodes) $V = \{1, 2, \ldots, n\}$.
- → A set of **edges** $E$ connecting pairs of vertices.

**Example:** $V = \{1, 2, 3, 4, 5, 6\}$,

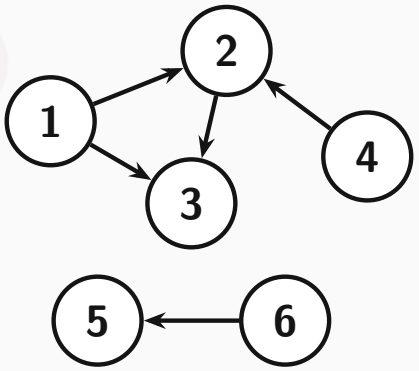
$E = \{(1, 2), (1, 3), (2, 3), (4, 2), (6, 5)\}$.

- → **Undirected:** edges are two-way $\{u, v\}$.
- → **Directed:** edges are one-way arrows $(u \to v)$.

**Undirected graph**




**Directed graph**



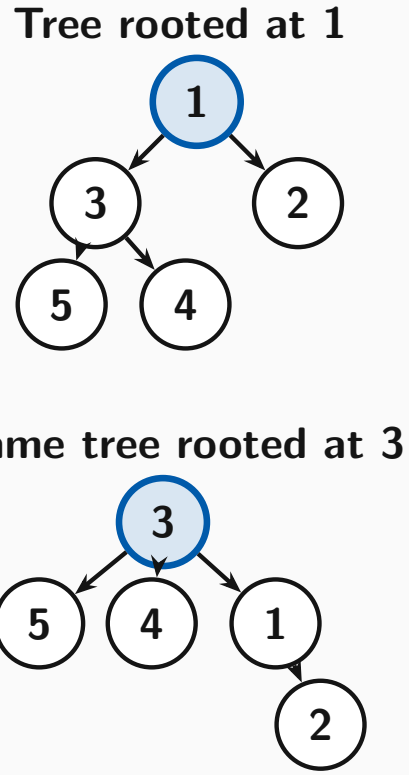
## Paths, Cycles, and Degrees

- **Path:** A sequence of vertices $(v_1, v_2, \ldots, v_k)$ with an edge between each consecutive pair $(v_i, v_{i+1})$. A path is **simple** if all vertices are distinct.
- **Cycle:** A path of length $\geq 3$ where $v_1 = v_k$ and $v_1, \ldots, v_{k-1}$ are distinct.
- **Degree** $\deg(v)$**:** The number of edges incident to vertex $v$.
  - In directed graphs, we split into **in-degree** $\deg^-(v)$ and **out-degree** $\deg^+(v)$.
- **Handshaking Lemma:** Every undirected edge contributes 1 to the degree of both endpoints:

$$\sum_{v \in V} \deg(v) = 2|E|.$$

# Connectivity, Trees, and Rooted Trees

- → **Connected Graph:** A path exists between any two vertices $u, v \in V$.
  - → Maximal connected subgraphs are called **connected components**.
- → **Tree:** A connected, acyclic undirected graph.
  - → Equivalent: connected with $|E| = |V| - 1$.
  - → Equivalent: **unique** simple path between any two vertices.
- → **Rooted Tree:** Fixing one special **root** node $r$ orients all edges as parent → child.





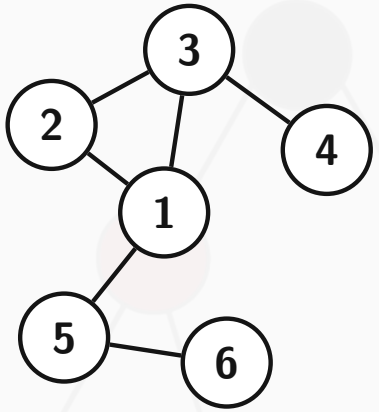
# Storing Graphs: Representations and Trade-Offs

# Three Ways to Represent a Graph

**1. Edge List**

Store $E$ as pairs $(u, v)$:

```
2 3
1 2
3 1
3 4
1 5
6 5
```




**2. Adjacency List**

For each $u$, list neighbours:

```
1:  2, 3, 5
2:  3, 1
3:  2, 1, 4
4:  3
5:  1, 6
6:  5
```

**3. Adjacency Matrix:** An $N \times N$ table $A$ where $A[u][v] = 1$ if $(u, v) \in E$, else 0.

# Adjacency List in Code (C++ and Python)

## C++ (vector<int>)

```cpp
const int MAXN = 100005;
vector<int> g[MAXN];

int main() {
  int n, m;
  cin >> n >> m;
  for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u); // 2-way
  }
}
```

## Python (list of lists)

```python
import sys
input = sys.stdin.readline

n, m = map(int, input().split())
g = [[] for _ in range(n + 1)]
for _ in range(m):
  u, v = map(int, input().split())
  g[u].append(v)
  g[v].append(u) # undirected
```

Total integers stored across all lists: $2|E|$ (or $|E|$ if directed).

# Adjacency Matrix in Code

```
const int MAXN = 1005;
bool g[MAXN][MAXN];

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        g[u][v] = true;
        g[v][u] = true; // undirected
    }
}
```

**Matrix $g[u][v]$ for the 6-node graph**

| | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 | 1 | 0 |
| 2 | 1 | 0 | 1 | 0 | 0 | 0 |
| 3 | 1 | 1 | 0 | 1 | 0 | 0 |
| 4 | 0 | 0 | 1 | 0 | 0 | 0 |
| 5 | 1 | 0 | 0 | 0 | 0 | 1 |
| 6 | 0 | 0 | 0 | 0 | 1 | 0 |

Symmetric ($g[u][v] = g[v][u]$) when undirected.

# Comparison: Memory and Operations

## Memory ($N = |V|$, $E = |E|$):

| **Structure** | $N{=}10^3, E{=}10^3$ | $N{=}10^3, E{=}10^5$ | $N{=}5000, E{=}2{\cdot}10^5$ | $N{=}10^5, E{=}2{\cdot}10^5$ |
|---|---|---|---|---|
| **Adj. List** $O(N{+}E)$ | $\approx$0.01 MB | $\approx$1 MB | $\approx$2 MB | $\approx$2 MB |
| **Adj. Matrix** $O(N^2)$ | $\approx$1 MB | $\approx$1 MB | $\approx$25 MB | $\approx$10 **GB** |

## Operation Time Complexities:

| **Operation** | **Adjacency List** | **Adjacency Matrix** | **Edge List** |
|---|---|---|---|
| Does edge $(u, v)$ exist? | $O(\deg(u))$ | $O(1)$ | $O(E)$ |
| Iterate neighbours of $u$ | $O(\deg(u))$ | $O(N)$ | $O(E)$ |
| Inspect all edges | $O(N+E)$ | $O(N^2)$ | $O(E)$ |
| Add / delete edge $(u, v)$ | $O(1)$ / $O(\deg(u))$ | $O(1)$ / $O(1)$ | $O(1)$ / $O(E)$ |
| Add / delete vertex $u$ | $O(1)$ / $O(N+E)$ | $O(N)$ / $O(N)$ | $O(1)$ / $O(E)$ |

# Try It Yourself: Representations and Degrees

Suppose an undirected simple graph has $N = 100\,000$ vertices and $M = 150\,000$ edges.

1. What is the sum of $\deg(v)$ over all 100 000 vertices?
2. If you store the graph in an adjacency list `g[1..N]`, how many total loop iterations does

   $$\texttt{for } u = 1 \ldots N: \quad \texttt{for } v \text{ in } g[u]: \ldots$$

   perform, and why is this much faster than scanning an adjacency matrix?
3. Can an undirected simple graph on 5 vertices have vertex degrees $3, 3, 3, 2, 2$?

*Discuss with a neighbour; full solutions are at the end of the slides.*

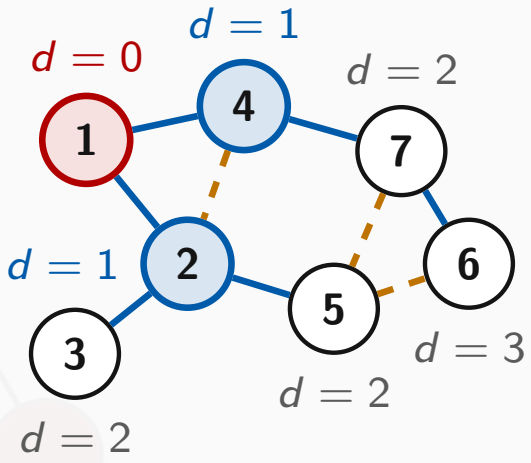
# Breadth-First Search (BFS) and Unweighted Shortest Paths



# Breadth-First Search (BFS): Layer-by-Layer Exploration

**Core Idea:**

- → Start at source vertex $s$ (layer 0).
- → Visit all neighbours at distance 1, then distance 2, then distance 3, and so on.
- → Maintain a **FIFO queue** $q$ of discovered vertices whose neighbours still need to be inspected.
- → Mark a vertex `visited` **as soon as it is pushed** into $q$ so it is never enqueued twice.




Solid blue: **BFS tree edges**; dashed: **cross edges**.

# BFS Step-by-Step Walkthrough

Using the adjacency list from the previous slide starting at vertex 1:

| **Pop** $v$ | **List** $g[v]$ | **Newly Discovered** ($d[u] = d[v] + 1$) | **Queue** $q$ |
|---|---|---|---|
| Init | — | Push 1 ($d[1] = 0$) | [1] |
| 1 | 4, 2 | Push 4, 2 ($d = 1$, par $= 1$) | [4, 2] |
| 4 | 1, 2, 7 | 1, 2 already visited; push 7 ($d = 2$, par $= 4$) | [2, 7] |
| 2 | 1, 3, 5, 4 | Push 3, 5 ($d = 2$, par $= 2$) | [7, 3, 5] |
| 7 | 4, 5, 6 | Push 6 ($d = 3$, par $= 7$) | [3, 5, 6] |
| 3, 5, 6 | all visited | None | [] (done) |

→ Discovery order: **1**, **4**, **2**, **7**, **3**, **5**, **6**, with distances $d = [0, 1, 1, 2, 2, 2, 3]$.

# BFS with Shortest Paths and Parent Reconstruction

## C++ Implementation

```
vector<int> dist(n + 1, -1);
vector<int> parent(n + 1, 0);
queue<int> q;
q.push(start);
dist[start] = 0;

while (!q.empty()) {
  int v = q.front();
  q.pop();
  for (int u : g[v]) {
    if (dist[u] == -1) {
      dist[u] = dist[v] + 1;
      parent[u] = v;
      q.push(u);
    }
  }
}
```

## Properties of BFS:

- → Tree edges form a **shortest-path tree** rooted at `start`.
- → Every vertex is enqueued at most once and each edge is inspected twice: $\mathbf{O(N + E)}$ time and space.
- → **Path reconstruction** (*Message Route*): follow `cur = parent[cur]` from target $t$ back to `start`, then reverse.

## Try It Yourself: BFS Shortest Path

Consider the 7-vertex graph from the BFS walkthrough (edges: $(1,4),(1,2),(4,2),(4,7),(2,3),(2,5),(7,5),(7,6),(5,6)$).

1. What is the shortest distance `dist[6]` from vertex 1 to vertex 6, and what is the parent chain `6 -> parent[6] -> ... -> 1`?
2. Why is the edge $(5,6)$ a **cross edge** rather than a tree edge when 7 is popped before 5?
3. What bug occurs if you set `visited[u] = true` when *popping* $u$ from the queue instead of when *pushing* $u$?

# Depth-First Search (DFS) and Edge Classification



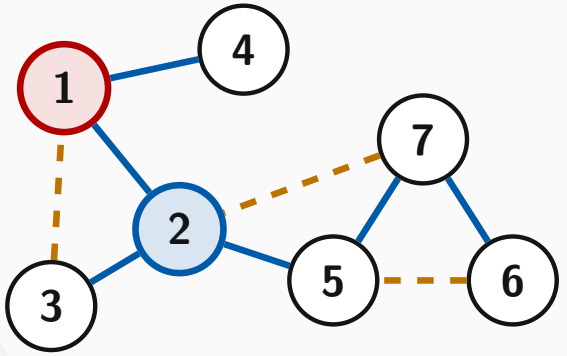
# Depth-First Search (DFS): Plunge and Backtrack

**Core Idea:**

→ Follow one branch as deep as possible until stuck, then **backtrack**.

→ Naturally expressed via recursion (or an explicit LIFO stack).

```
void dfs(int v) {
    vis[v] = true;
    tin[v] = ++timer;
    for (int u : g[v]) {
        if (!vis[u]) {
            dfs(u);
        }
    }
    tout[v] = ++timer;
}
```




**DFS Entry (`tin`) order:**

$1, 4, 2, 3, 5, 7, 6$

**DFS Exit (`tout`) order:**

$4, 3, 6, 7, 5, 2, 1$

# DFS Properties and Edge Classification

- → **Subtree Property:** Once DFS exits vertex $v$, it has visited *every* vertex reachable from $v$ along unvisited paths. Vertex $u$ is in $v$'s DFS subtree iff $\text{tin}[v] \leq \text{tin}[u] \leq \text{tout}[u] \leq \text{tout}[v]$.
- → **Edge Types in Undirected Graphs:** Every edge is either:
  1. a **Tree Edge** (discovers a new vertex $u$ from $v$), or
  2. a **Back Edge** (connects $v$ to an already-visited ancestor $u \neq \text{parent}[v]$).
- → **Edge Types in Directed Graphs:** In addition to tree and back edges, directed graphs can have **forward edges** (to a proper descendant) and **cross edges** (to a previously finished branch).
- → **Complexity:** Visits every vertex and edge once: $\mathbf{O(N + E)}$ time and $\mathbf{O(N)}$ recursion stack.

# Try It Yourself: DFS Trace and Back-Edges

In the 7-vertex DFS walkthrough graph, the adjacency lists are:

$$1 : [4, 2, 3], \quad 2 : [1, 3, 5, 7], \quad 3 : [2, 1], \quad 4 : [1],$$
$$5 : [2, 7, 6], \quad 6 : [7, 5], \quad 7 : [2, 5, 6].$$

1. When DFS is at vertex 3 (called from 2, which was called from 1), it inspects neighbour 1. Why does $(3, 1)$ prove the existence of a simple cycle?
2. List all three back-edges encountered during `dfs`$(1)$ and the simple cycle each back-edge closes.

# Core Algorithmic Patterns for the Problem Sheet



## Pattern 1: Finding Connected Components

- → Loop through vertices $i = 1, \ldots, n$.
- → Whenever $i$ is unvisited, start DFS or BFS from $i$. This traversal visits exactly the vertices in $i$'s connected component.
- → Record $i$ as that component's representative and continue scanning.

**Application — Building Roads:** If the graph has $C$ components represented by $r_1, \ldots, r_C$, add roads

$$(r_1, r_2), (r_2, r_3), \ldots, (r_{C-1}, r_C).$$

Each road joins two components, so $C - 1$ roads are sufficient and necessary.

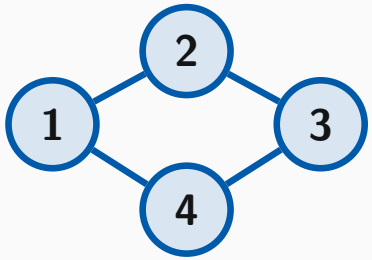
**Complexity:** $O(N + E)$ time with an adjacency list.

# Pattern 2: Recognising Cyclic Components

**Theorem:** A connected component of a simple undirected graph is a simple cycle if and only if every vertex in it has degree 2.

→ A cycle gives every vertex exactly two incident edges.

→ Conversely, in a finite connected component where every vertex has degree 2, following edges must eventually return to the start. Connectivity rules out vertices outside that cycle.



Every vertex has degree 2

# Pattern 2: Applying the Degree Test

**Goal:** Count connected components that are simple cycles.

1. For each unvisited vertex, traverse its entire component.
2. Check whether every vertex in the component has degree 2.
3. Count the component if and only if that check succeeds.

The traversal ensures the component is connected; the degree test then characterises a cycle.

**Complexity:** $O(N + E)$ for the whole graph.

# Pattern 3: Finding Cycles in Undirected Graphs

**Application — Round Trip:** During DFS, track each vertex's parent.

1. If an edge from $v$ discovers an unvisited neighbour $u$, set $\texttt{parent}[u] = v$ and continue DFS.
2. Ignore the edge back to $\texttt{parent}[v]$; it is just the same undirected edge in reverse.
3. If $u$ is already visited and $u \neq \texttt{parent}[v]$, the edge $(v, u)$ closes a cycle with the tree path from $u$ to $v$.

Reconstruct the cycle by following parent pointers from $v$ back to $u$.

Traversal and reconstruction take $O(N + E)$ time.

# Pattern 4: Functional Graphs

**Definition:** A functional graph is a directed graph where every vertex has exactly one outgoing edge, $u \to f(u)$.

- → Following edges from any starting vertex eventually reaches a directed cycle.
- → Each component consists of one directed cycle with trees feeding into its cycle vertices (a $\rho$-shape).
- → **Badge:** Repeatedly follow the successor until a vertex is visited twice; that repeated vertex is the first student receiving a second hole.
- → **Game on Axis:** Treat out-of-bounds jumps as edges to a terminal sink 0. In the reversed graph, the vertices that terminate form a tree rooted at 0.

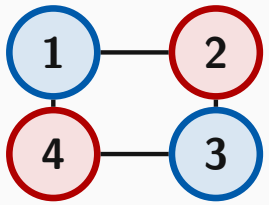
# Pattern 5: Bipartite Graphs

**Definition (Building Teams):** A graph is **bipartite** if its vertices can be split into two teams so every edge connects opposite teams.

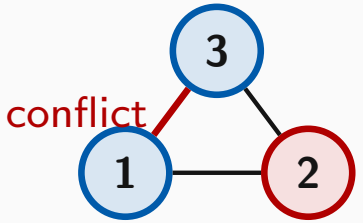
**Theorem:** A graph is bipartite if and only if it has no odd-length cycle.

- → Even cycles alternate colours consistently.
- → An odd cycle eventually requires one edge to connect vertices of the same colour.

**Bipartite: even cycle $C_4$**




**Not bipartite: odd cycle $C_3$**



# Pattern 6: 2-Colouring with BFS or DFS

1. For each uncoloured vertex $s$, set $\texttt{col}[s] = 1$ and start BFS/DFS.
2. For each edge $(v, u)$:
   - → If $\texttt{col}[u] = 0$, assign the opposite colour: $\texttt{col}[u] = 3 - \texttt{col}[v]$.
   - → If $\texttt{col}[u] = \texttt{col}[v]$, the graph is not bipartite; output `IMPOSSIBLE`.
3. Repeat from every uncoloured vertex so disconnected components are also checked.

**Complexity:** $O(N + E)$ time and $O(N + E)$ space with an adjacency list.

## Pattern 7: Tree Diameter in Two Traversals

**Problem (Tree Diameter):** Find the maximum distance between any two nodes in a tree, without dynamic programming.

1. Pick an arbitrary vertex $r$ and run BFS/DFS to find a farthest vertex $u$.
2. Run a second BFS/DFS from $u$ to find a farthest vertex $v$.
3. The distance $\text{dist}(u, v)$ is the tree diameter.

**Why it works:** Paths in a tree are unique. The farthest vertex from an arbitrary root is an endpoint of a longest path; the second traversal reaches the other endpoint.

**Complexity:** Two linear traversals take $O(N)$ time and $O(N)$ space.

# Pattern 8: Topological Sorting a DAG

A **Directed Acyclic Graph (DAG)** has no directed cycles. Its vertices can be ordered so every edge $u \to v$ goes forward in the order.

**Kahn's Algorithm (BFS):**

1. Compute the in-degree `indeg`$[v]$ of every vertex.
2. Add every vertex with in-degree 0 to a queue.
3. Pop $u$, append it to the order, and decrement the in-degree of each outgoing neighbour $v$.
4. Whenever `indeg`$[v]$ reaches 0, add $v$ to the queue.

Each vertex and edge is processed once, so the algorithm takes $O(N + E)$ time. If fewer than $N$ vertices are output, the graph contains a directed cycle.

## Pattern 9: Dynamic Programming on a DAG

A topological order ensures that when a vertex is processed, all contributions from its predecessors have already been calculated.

**Counting paths (DAG Number of Routes):** Set $\mathrm{dp}[1] = 1$. For each edge $u \to v$ in topological order, add $\mathrm{dp}[u]$ to $\mathrm{dp}[v]$ modulo $10^9 + 7$.

$$\mathrm{dp}[v] = \sum_{u \to v} \mathrm{dp}[u] \pmod{10^9 + 7}.$$

**Longest path (AtCoder DP G):** Process vertices in reverse topological order and use

$$\mathrm{dp}[u] = \max_{u \to v}(1 + \mathrm{dp}[v]),$$

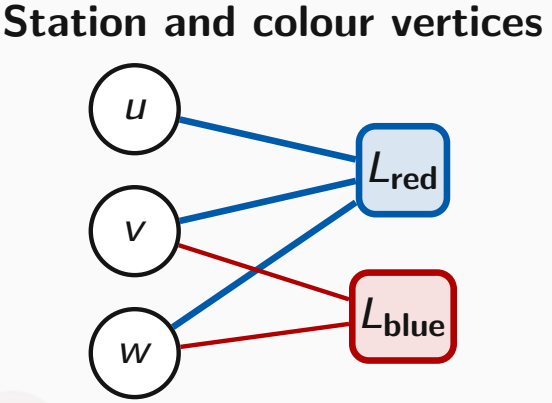
with $\mathrm{dp}[u] = 0$ for a sink. Both algorithms run in $O(N + E)$ time.

## Pattern 10: Auxiliary Graph Modelling (Rudolf and Subway)

**Challenge:** Riding tracks of the same colour costs one ticket. Joining every pair of stations on one colour directly can create $O(N^2)$ edges.

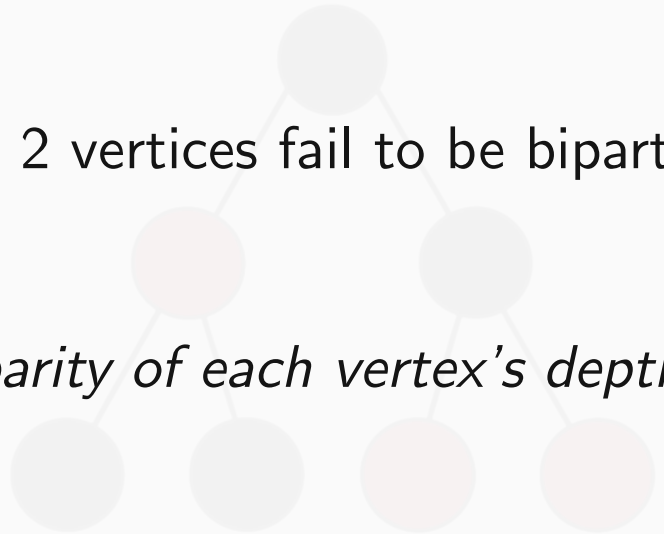
**Star expansion:**

- → Create an auxiliary vertex $L_c$ for each colour $c$.
- → Replace every track $(u, v, c)$ by edges $u - L_c$ and $v - L_c$.
- → Riding one line from station $u$ to station $v$ corresponds to $u \to L_c \to v$.




Run BFS from $b$ to $e$ in the expanded graph; the answer is $\text{dist}(b, e)/2$. The expanded graph has $O(N + M)$ vertices and edges.

## Try It Yourself: Tree Bipartiteness



Can a tree with $N \geq 2$ vertices fail to be bipartite? Explain why or why not.

*Hint: Consider the parity of each vertex's depth from a chosen root.*

# Try It Yourself: Counting Paths in a DAG

Suppose a DAG has vertices $\{1, 2, 3, 4\}$ and directed edges

$$1 \to 2, \quad 1 \to 3, \quad 2 \to 3, \quad 2 \to 4, \quad 3 \to 4.$$

Find a valid topological ordering and calculate the number of directed paths from vertex 1 to vertex 4.

## Try It Yourself: Longest Path in a DAG

Using the same DAG, find the length (number of edges) of the longest directed path from vertex 1 to vertex 4.

*Hint: Compare the direct routes with the route that passes through both vertices 2 and 3.*



# Conclusion

# Summary: Matching Problems to Graph Techniques

| Problem Goal | Key Structure / Invariant | Algorithm |
|---|---|---|
| Connect components | Reachability equivalence classes | DFS/BFS, $O(N+E)$ |
| Shortest unweighted route | Layer-by-layer distance | BFS + `parent[]`, $O(N+E)$ |
| Find undirected cycle | Back-edge to $u \neq$ `parent[`$v$`]` | DFS + `parent[]`, $O(N+E)$ |
| Split into two teams | Bipartite $\iff$ no odd cycle | 2-colouring, $O(N+E)$ |
| Component is a cycle | Connected and $\forall v, \deg(v) = 2$ | Degree check, $O(N+E)$ |
| Paths / longest in DAG | Acyclic $\implies$ topological order | Kahn / DFS DP, $O(N+E)$ |
| Tree diameter (no DP) | Farthest node is diameter end | Two-BFS/DFS, $O(N)$ |
| Same-colour line rides | Star expansion with nodes $L_c$ | Auxiliary BFS, $O(N+E)$ |

# Try It Yourself: Representations and Degrees (Answers)

1. **Sum of degrees:** By the Handshaking Lemma,

$$\sum_{v=1}^{100\,000} \deg(v) = 2M = 2 \times 150\,000 = 300\,000.$$

2. **Adjacency list loop iterations:** The inner loop runs $\deg(u)$ times for vertex $u$, so total iterations equal $\sum_u \deg(u) = 300\,000$ ($3 \times 10^5$ operations), compared to $N^2 = 10^{10}$ operations in an adjacency matrix.
3. **Degree sequence** $3, 3, 3, 2, 2$**:** Impossible! Their sum is $3 + 3 + 3 + 2 + 2 = 13$, which is odd, violating $\sum \deg(v) = 2|E|$.

# Try It Yourself: BFS Shortest Path (Answers)

1. **Shortest path to** 6**:** `dist`$[6] = 3$. Following parents gives $6 \leftarrow 7 \leftarrow 4 \leftarrow 1$, so the shortest path is $1 \rightarrow 4 \rightarrow 7 \rightarrow 6$.
2. **Why** $(5, 6)$ **is a cross edge:** Both 7 and 5 are in layer $d = 2$, with 7 enqueued before 5. When 7 is popped, it discovers 6 ($d[6] = 3$). Later, when 5 is popped, 6 is already visited, so $(5, 6)$ is a non-tree cross edge.
3. **Marking visited on pop instead of push:** The same vertex can be pushed into the queue multiple times by different neighbours in the previous layer, causing duplicate work and up to $O(E)$ queue blowup (and incorrect parent overwrites).

# Try It Yourself: DFS Trace and Back-Edges (Answers)

1. **Why** $(3, 1)$ **proves a cycle:** At vertex 3, $\texttt{parent}[3] = 2$. Neighbour 1 is already visited and $1 \neq \texttt{parent}[3]$, so 1 is an ancestor of 3 in the DFS tree. Combining the tree path $1 \to 2 \to 3$ with the back-edge $(3, 1)$ gives the simple cycle $1 - 2 - 3 - 1$.
2. **All three back-edges:**
   - $\to$ $(3, 1)$ closes cycle $1 - 2 - 3 - 1$.
   - $\to$ $(7, 2)$ closes cycle $2 - 5 - 7 - 2$.
   - $\to$ $(6, 5)$ closes cycle $5 - 7 - 6 - 5$.

# Try It Yourself: Tree Bipartiteness (Answer)

**Yes.** Root the tree anywhere and colour each vertex by the parity of its depth.

Every tree edge joins a parent to a child, whose depths differ by exactly one, so its endpoints have opposite colours. Thus this is a valid 2-colouring. Equivalently, a tree has no cycles, and hence no odd cycles.

# Try It Yourself: Counting DAG Paths (Answer)

A valid topological ordering is $[1, 2, 3, 4]$. Initialise $\mathrm{dp}[1] = 1$ and process the edges in that order:

$$\begin{aligned}\mathrm{dp}[2] &= 1,\\ \mathrm{dp}[3] &= \mathrm{dp}[1] + \mathrm{dp}[2] = 2,\\ \mathrm{dp}[4] &= \mathrm{dp}[2] + \mathrm{dp}[3] = 1 + 2 = \mathbf{3}.\end{aligned}$$

The three routes are $1 \to 2 \to 4$, $1 \to 3 \to 4$, and $1 \to 2 \to 3 \to 4$.

# Try It Yourself: Longest DAG Path (Answer)

The longest route is $1 \to 2 \to 3 \to 4$, which contains **3 edges**.

The other routes are $1 \to 2 \to 4$ and $1 \to 3 \to 4$, each with 2 edges. Therefore the longest-path answer is 3.